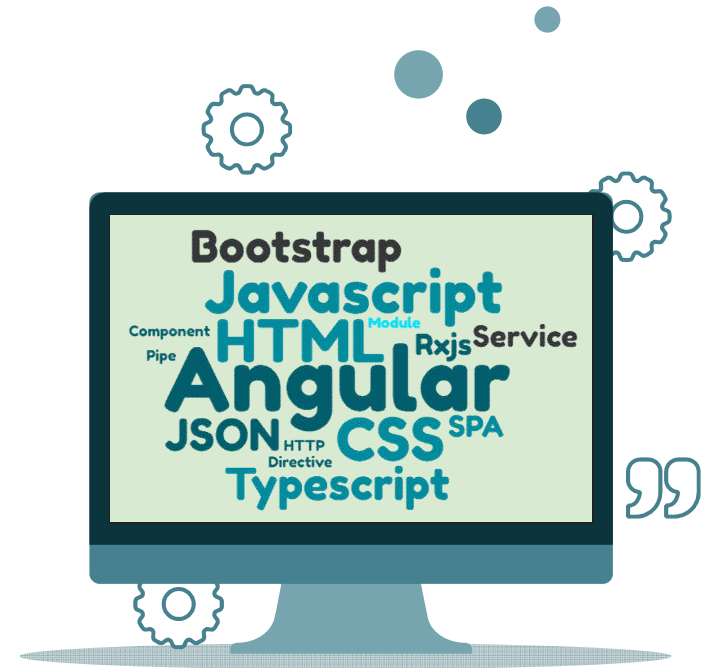


02

Introduction à Angular





Chapitre 2 Introduction à Angular

01

Présentation d'Angular

03

TypeScript :Memo

02

Angular Vs EcmaScript6+
Vs TypeScript



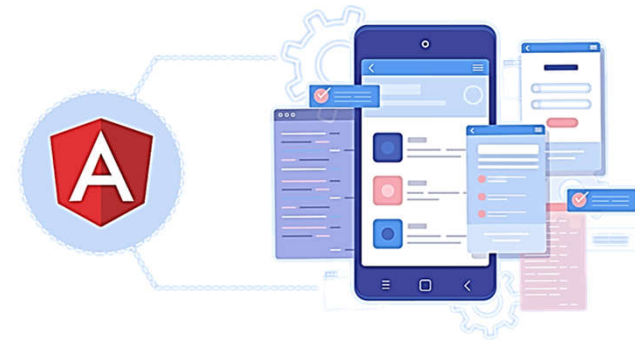
01

Présentation d'Angular



Qu'est ce qu'Angular ?

- **Angular** est un framework frontend de développement d'applications Web de type SPA (Single Page Application) et mobile (Cross plateforme),
- Il est open-source et développé par Google,
- Il est conçu pour faciliter la création d'applications Web complexes en offrant une structure solide et des fonctionnalités avancées pour :
 - la gestion des données,
 - la navigation,
 - le routage,
 - et la mise en œuvre de l'interface utilisateur.
- Angular utilise les langages suivants:
 - **HTML** pour les vues,
 - **CSS** pour les styles,
 - **TypeScript** pour les scripts.



Histoire d'Angular



▪ Angular 1 (AngularJS)

- Première version de Angular qui était très populaire,
- Développée dans les locaux de **Google en 2009**,
- Les applications Angular JS sont écrites en **JS** et basées sur une architecture **MVC / MVVM** coté client.

▪ Angular 2 (Angular)

- 1ère version beta annoncée en **octobre 2014**,
- Reprend certains concepts d'AngularJS (ex. le pattern design **MVVM**),
- Basé sur **TypeScript** et la **Programmation Orientée Composant**,
- Version officielle sortie en **2016**,
- **Angular 4,5,6,7,8 ,9 ,10 ,.... 18** sont des simples mises à jours de **Angular 2** avec des améliorations au niveau de performance.

Historique des versions d'Angular

- **Angular** sort une nouvelle version majeure tous **les 6 mois environ** :
 - Angular 2 est sorti en septembre 2016,
 - Angular 4 en mars 2017,
 - Angular 5 en nov. 2017,
 - Angular 6 en mai 2018,
 - Angular 7 en octobre 2018,
 - Angular 8 en mai 2019,
 - Angular 9 en février 2020,
 - Angular 10 en juin 2020,
 - Angular 11 en nov. 2020,
 - Angular 12 en mai 2021,
 - Angular 13 en nov. 2021,
 - Angular 14 en juin 2022,
 - **Angular 15 en nov. 2022 (LTS),**
 - **Angular 16 en mai 2023 (LTS),**
 - **Angular 17 en nov. 2023 (Active),**
 - **Angular 18 en Mai 2024 (Latest)**

Pourquoi choisir Angular ?

- **Environnement structuré et cohérent :**
 - Développement rapide et fiable des applications web.
 - Structuration de projet claire pour une meilleure maintenabilité.
- **Utilisation de TypeScript :**
 - Avantages de la programmation orientée objet.
 - Vérification statique du code pour minimiser les erreurs.
- **Outils et fonctionnalités avancées :**
 - Gestion efficace des données et de la logique de l'application.
 - Création d'interfaces utilisateur complexes et réactives.
- **Communauté et ressources étendues :**
 - Forte communauté de développeurs.
 - Abondance de ressources en ligne pour l'apprentissage et le support.

Points faibles d'angular ?

- **Complexité pour les débutants :**
 - Courbe d'apprentissage raide pour les développeurs débutants.
 - Nécessité de maîtriser de nombreux concepts avant de devenir efficace.
- **Taille du framework :**
 - Angular est un framework volumineux.
 - Temps de chargement plus longs et impact sur les performances des applications.
- **Approche structurée et inflexible :**
 - Approche rigide pour la structuration des applications.
 - Peut ne pas convenir à certains projets ou approches de développement plus flexibles.
- **Évolution constante :**
 - Changements de comportement entre les versions.
 - Efforts de mise à jour importants pour les projets existants.

Angular : Caractéristiques clés (1/2)

Injection des dépendances

- Concept connu en programmation orientée objet
- Utilisé par plusieurs frameworks Back-End (Spring, Symfony...)
- Consiste à **utiliser des classes sans faire de l'instanciation statique**

Dom Virtuel

- Introduit par **React**
- Une **représentation en mémoire** du DOM physique
- Les modifications se font sur ce DOM virtuel ensuite **Angular** s'occupe de les synchroniser vers le DOM physique.

Angular : Caractéristiques clés (2/2)

Change Detector

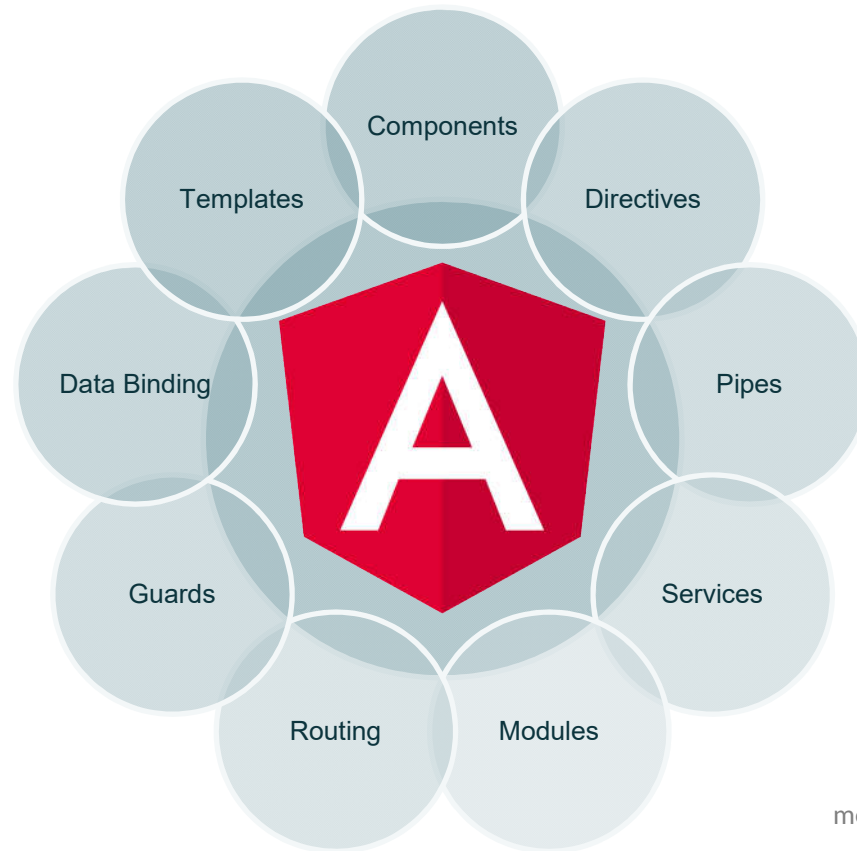
- Réalisé par la librairie **zone.js**
- Utilisé pour synchroniser la vue avec le composant,
- Le change detector est un module d'Angular qui permet de détecter les modifications apportées aux données du composant et de mettre à jour le template du composant (la vue) en conséquence.
- Le change detector parcourt l'arborescence de données du composant et compare les valeurs actuelles avec les valeurs précédentes pour déterminer si des modifications ont été apportées.
- Si des modifications sont détectées, le change detector met à jour le template du composant en conséquence.
- Le change detector est exécuté de manière asynchrone, de sorte qu'il n'interrompt pas l'exécution de l'application pendant qu'il met à jour le template du composant.

Angular: Environnement de travail

Outils

- **npm** (node package manager) : le [gestionnaire de paquets](#) par défaut pour une application JavaScript
- **angular-cli** (command line interface) : outil proposé par Google pour faciliter la construction d'une application Angular [en exécutant directement des commandes](#)
- **webpack** : bundler (package) JavaScript
 - un bundle : un fichier qui contient la totalité du code et qui sera utilisé en production
 - Webpack est par définition un static module bundler. Son rôle est d'analyser tous les imports dans le code d'une application (import ES6, require, voire même les imports réalisés dans les feuilles de styles) , construit le graphe de dépendances, regroupe des ressources de même nature (.js ou .css...) et de tous les réunir pour construire votre bundle.
- **Ivy** : le [moteur de compilation](#) et de rendu d'Angular intégré partiellement depuis la version 8 : intégralement (version 9) : permettant d'accélérer la compilation et une meilleure lisibilité des messages d'erreur.

Angular: Vocabulaire



02

Angular vs EcmaScript6+ vs Type Script



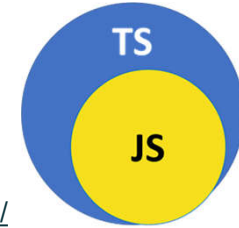
Angular et ECMAScript 6+

- **ECMAScript 6** est une version **JavaScript** approuvée par l'organisme de normalisation **ECMA** en 2015,
- On désigne par **ES6+** : Toutes les versions qui suivent ES6
- **ES6**, via sa nouvelle syntaxe, apporte plusieurs nouvelles évolutions à JavaScript et notamment à une application Angular, les nouveaux mots clés sont :
 - **class** : permet de créer des classes
 - **let** : permet de déclarer une variable locale, dans le contexte où elle a été assignée.
 - **const** : permet de déclarer des constantes (**const PI = 3.141592 ;**)
 - **Les fonctions fléchées** : syntaxe (paramètre) => { corps de la fonction }
 - Très utilisé dans la programmation asynchrone qui nécessite beaucoup de fonctions anonymes

Angular et TypeScript



<https://www.typescriptlang.org/>



- **TypeScript** est un langage de programmation libre et open-source, développé et présenté par Microsoft en 2012,
- **TypeScript** est une surcouche d'ECMAScript permettant l'ajout optionnel de typage statique,
- Le code **TypeScript** est transcompilé en JavaScript, pouvant ainsi être interprété par n'importe quel navigateur web ou moteur JavaScript : `tsc mon_code.ts → mon_code.js`
Attention, l'extension des fichiers TypeScript est .ts et non .js !
- Principaux ajouts par rapport à JavaScript :
 - Typage fort → facilite les débogages
 - Notions orientées objet : interfaces, etc
 - Utilise les décorateurs (les annotations)
 - Erreurs à la compilation

Angular est développé avec TypeScript

De l'ECMAScript au TypeScript

- L'**ECMAScript** est **quasiment** du TypeScript valide étant donné que le typage est optionnel,
- Il est donc facile de migrer progressivement de l'**ECMAScript** vers le TypeScript,
- L'un des principaux changements empêchant l'**ECMAScript** d'être du TypeScript valide est la nécessité de déclarer les propriétés d'une classe,

```
class Customer {  
  constructor(firstName, lastName) {  
    this.firstName = firstName;  
    this.lastName = lastName;  
  }  
}
```

Unresolved variable lastName more... (%F1)

```
1 $ tsc customer.ts  
2 customer.ts(11,18): error TS2339: Property 'firstName' does not exist on type 'Customer'.  
3 customer.ts(12,18): error TS2339: Property 'lastName' does not exist on type 'Customer'.
```

- La bonne pratique est de déclarer les propriétés avant le constructeur.

```
1 class Customer {  
2   firstName : string;  
3   lastName : string;  
4   constructor( firstName : string, lastName : string){  
5     this.firstName = firstName;  
6     this.lastName = lastName;  
7   }  
8 }
```


03

Type Script : Memo



TypeScript: Cheat sheet

```
1 // Typescript Cheat Sheet : primitive Types
2
3 // let better than var more scope and call strict.
4 // Use const for variables and readonly for properties
5
6 // boolean:
7 let isDone: boolean = false
8 // number:
9 let value: number = 6 // or hexadecimal, binary, octal
10 value = 0xf00d
11 value = 0b1010
12 value = 0o744
13 // string:
14 let someStr: string = 'Something\n'
15 let fname: string = 'Adam'
16 let age: number = 30
17 let str: string = someStr + fname + (age + 1)
18 // typeof: like javascript
19 let x: number = 10
20 if (typeof x == 'number') {
21     // x is a number
22 }
```

TypeScript: Cheat sheet

```
1 // Typescript Cheat Sheet : basic Types
2
3 // any:
4 let n: any = 4;
5 n = "str";
6 n = false;
7 let an: any[];
8 // unknown:
9 let unknownValue: unknown = 10;
10 if (typeof unknownValue === 'number') {
11     // now we know it's a number
12 }
13 // special: undefined; null;
14 let specialValue: undefined = undefined;
15 let nullValue: null = null;
16 // never:
17 function err(msg: string): never {
18     throw new Error(msg);
19 }
20 // type assertions:
21 let strval: unknown = "Hello";
22 let s: number = (<string>strval).length; //casts
```

TypeScript: Cheat sheet

```
1 // Typescript Cheat Sheet : Array, Tuple and enum
2
3 // array<>:
4 let list: number[] = [1, 2, 3];
5 let list2: Array<number> = [1, 2, 3];
6
7 // tuple:
8 let tuple: [string, number];
9 tuple = ["hello", 10];
10
11 // enum:
12 enum Color {Red, Green};
13 let c: Color = Color.Green;
14
15 // Union types:
16 let array: (number | string)[]; // Array can contain numbers or strings
17
18 // Assigning values to the array
19 array = [1, 2, 3]; // All numbers
20 array = ["a", "b", "c"]; // All strings
21 array = [1, "two", 3]; // Mixed types
```

TypeScript: Cheat sheet

```
1 // Typescript Cheat Sheet : Functions
2
3 function add(x:number, y:number): number { return x + y }
4
5 function example (defval:string="def", optionalval?: string) {
6     //do something
7 }
8 function params(fn:string, ...rest:string[]) {
9     //do something
10 }
11
12 // Arrow Functions
13 const square = (x: number): number => x * x;
14 console.log(square(5)); // Output: 25
15
16 const greetArrow = (name: string): void => console.log("Hello, " + name + "!");
17 greetArrow("Adam"); // Output: Hello, Adam!
18
19 // Immediately Invoked Function Expression (IIFE) with Parameters
20 ((name: string) => {
21     console.log("Hello, " + name + "!");
22 }("Adam"));
```

TypeScript: Cheat sheet

```
1 // Typescript Cheat Sheet : Classes
2
3 //members are public by default. can be individually set to private or protected.
4 //use readonly for constants.
5 class Example {
6     prop1: string;
7     static stprop: { x: 0, y: 0 };
8     constructor(msg: string) { this.prop1 = msg; }
9     move(dist: number) {
10         //do something
11     }
12     get prop1_accessor(): string { return this.prop1; }
13     set prop1_accessor(s: string) { this.prop1 = s; }
14 }
15 let exclass = new Example("Hello!");
16 class ExampleInherit extends Example {
17     constructor(msg: string) { super(msg); }
18     move(dist = 5) { super.move(dist); }
19 }
20 abstract class Test {
21     abstract func1(): void;
22     func2(): void {
23         //do something
24     }
25 } // Abstracts can be extended by classes of course
```

TypeScript: Cheat sheet

```
1 // Typescript Cheat Sheet : Interfaces
2
3 interface IExample {
4   label: string; // mandatory property
5   color?: string; // optional property
6   [propName: string]: any; // additional properties can have any name and type
7   iFunction(par1: string, par2: string): boolean; // function signature
8 }
9 // Implementing the Example interface
10 const exampleImplementation: IExample = {
11   label: "Example Label",
12   color: "red",
13   additionalProperty: 123, // Additional property
14   // Function signature implementation
15   iFunction(par1: string, par2: string): boolean {
16     console.log(par1, par2);
17     return true;
18   }
19 };
20 // Accessing properties and calling the function
21 console.log(exampleImplementation.label); // Output: Example Label
22 console.log(exampleImplementation.color); // Output: red
23 console.log(exampleImplementation.additionalProperty); // Output: 123
24 console.log(exampleImplementation.iFunction("arg1", "arg2")); // Output: arg1 arg2 true
25 interface IExampleExtend extends IExample, IExampleOther {...}
26 class Example implements IExample {...}
```

TypeScript: Cheat sheet

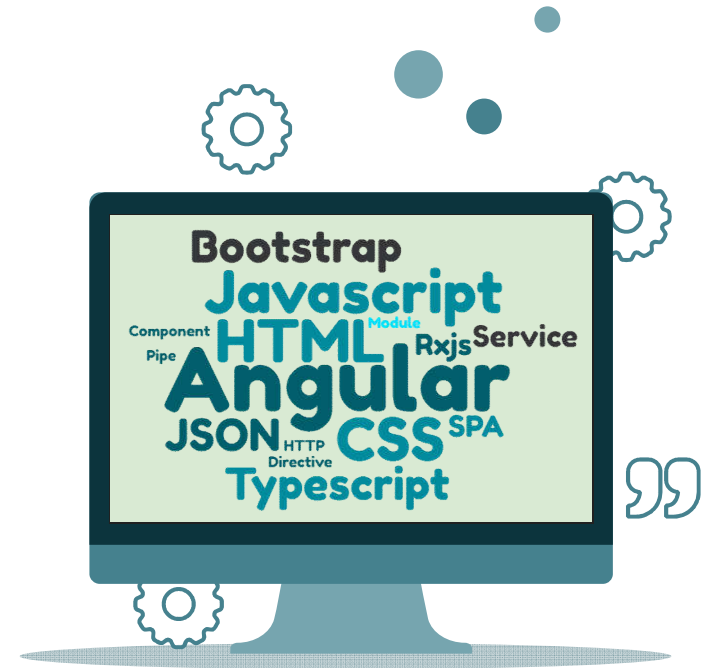
```
1 // Typescript Cheat Sheet : Modules
2
3 // In TypeScript, just like in ECMAScript 2015 (ES6),
4 // any file containing an import or export at its beginning is considered a module.
5 // Modules are executed in their own scope, not the global scope.
6 // This means that variables, functions, classes, etc. declared in a module are not visible outside the module
7 // unless they are explicitly exported using one of the export syntaxes.
8 // Conversely, to consume a variable, function, class, interface, etc. exported from a different module,
9 // it must be imported using one of the import syntaxes.
10 // @filename: maths.ts
11 export let pi = 3.14
12 export let squareTwo = 1.41
13 export const phi = 1.61
14 export class RandomNumberGenerator {}
15 export function absolute(num: number) {
16   return num < 0 ? num * -1 : num
17 }
18
19 // These can be used in another file via the import syntax
20
21 import { pi, phi, absolute } from "./maths.js"
22
23 console.log(pi)
24 const absPhi = absolute(phi)
```



Module TypeScript/JavaScript $\neq$ module Angular

03

Architecture du Framework





Chapitre 3 Architecture du Framework

01

Installation

03

Atelier 1

02

Création et structure de projet



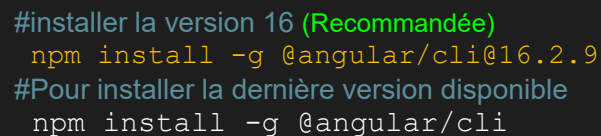
01

Installation



Etapes

- Télécharger et installer **NodeJS (18.4.0)**
 - <https://nodejs.org/fr/download/>
- Télécharger et Installer Angular CLI (<https://cli.angular.io/>)
 - Lancer l'invite de commandes sous Windows
 - Installer la **version 16**, pour cela taper la commande suivante :



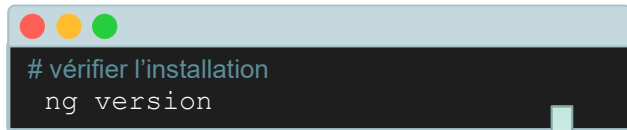
```
#installer la version 16 (Recommandée)
npm install -g @angular/cli@16.2.9
#Pour installer la dernière version disponible
npm install -g @angular/cli
```

Info

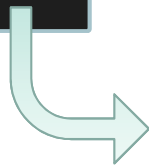
La commande `npm install -g @angular/cli` permet d'installer Angular sur votre ordinateur. La partie `npm install` indique à npm de télécharger et d'installer un paquet, tandis que `-g` indique que le paquet doit être installé globalement, c'est-à-dire qu'il sera disponible pour tous les projets sur votre ordinateur. Enfin, `angular/cli` est le nom du paquet Angular à installer.

Version d'Angular

- Redémarrer la console et puis vérifier la version :



```
# vérifier l'installation  
ng version
```



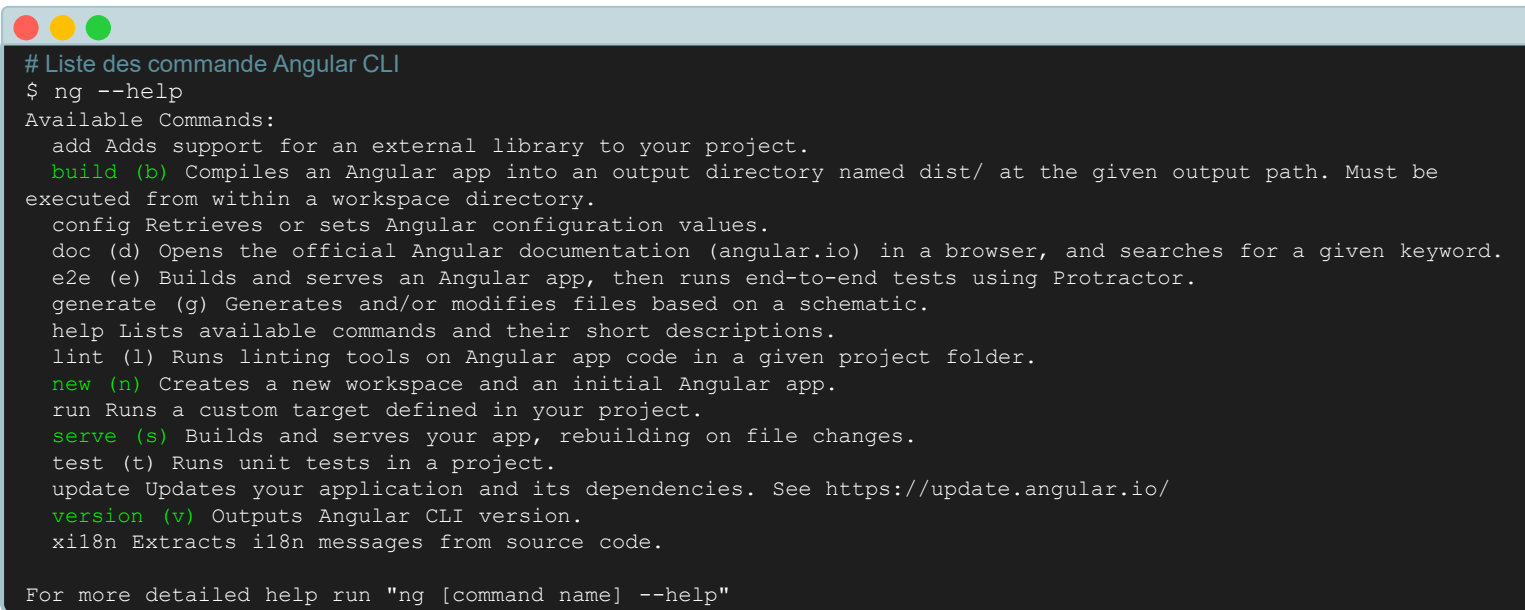
```
Angular CLI  
  
Angular CLI: 16.2.9  
Node: 18.14.0  
Package Manager: npm 9.3.1  
OS: win32 x64  
  
Angular:  
...  


| Package                    | Version             |
|----------------------------|---------------------|
| @angular-devkit/architect  | 0.1602.9 (cli-only) |
| @angular-devkit/core       | 16.2.9 (cli-only)   |
| @angular-devkit/schematics | 16.2.9 (cli-only)   |
| @schematics/angular        | 16.2.9 (cli-only)   |


```

Commande ng

- Pour explorer la liste des commandes Angular disponibles, exécuter la commande **ng --help**



```
# Liste des commande Angular CLI
$ ng --help
Available Commands:
  add Adds support for an external library to your project.
  build (b) Compiles an Angular app into an output directory named dist/ at the given output path. Must be
executed from within a workspace directory.
  config Retrieves or sets Angular configuration values.
  doc (d) Opens the official Angular documentation (angular.io) in a browser, and searches for a given keyword.
  e2e (e) Builds and serves an Angular app, then runs end-to-end tests using Protractor.
  generate (g) Generates and/or modifies files based on a schematic.
  help Lists available commands and their short descriptions.
  lint (l) Runs linting tools on Angular app code in a given project folder.
  new (n) Creates a new workspace and an initial Angular app.
  run Runs a custom target defined in your project.
  serve (s) Builds and serves your app, rebuilding on file changes.
  test (t) Runs unit tests in a project.
  update Updates your application and its dependencies. See https://update.angular.io/
  version (v) Outputs Angular CLI version.
  xil8n Extracts i18n messages from source code.

For more detailed help run "ng [command name] --help"
```

Environnement de développement

- Il y a plusieurs IDE qui supportent TypeScript et qui pourront nous aider lors de nos développements :
 - Visual Studio Code (recommandé par Microsoft pour les développements Angular, évidemment)
 - WebStorm (payant mais licence d'un an offerte si vous êtes étudiant) ;
 - Sublime Text avec un plugin à installer ;
- Il y en a plein d'autres, d'ailleurs le **site officiel de TypeScript** a listé sur sa page d'accueil les IDE recommandés pour TypeScript.
- Dans le cadre de notre formation, nous allons utiliser **Visual Studio Code** afin de réaliser les différents ateliers
- **Lien de téléchargement** : <https://code.visualstudio.com/>



Visual Studio Code

Quelles extensions utilisées sur VS Code ?

- Voici quelques extensions utiles pour Angular dans Visual Studio Code :

- **Angular Language Service** : Cette extension permet de bénéficier des services de langage Angular dans Visual Studio Code, tels que la vérification de syntaxe, la complétion de code, la détection d'erreurs, etc.
- **Angular Snippets** : Cette extension ajoute des snippets (fragments de code prédéfinis) pour Angular dans Visual Studio Code, ce qui permet de gagner du temps lors de l'écriture de code.
- **Path Intellisense** : Cette extension permet d'obtenir une complétion de code automatique pour les chemins de fichiers dans Visual Studio Code, ce qui peut être utile lorsque vous travaillez avec des projets Angular.



Angular Language Service v17.3.1
Angular [angular.io](#) | 6,550,226 | ★★★★★ (244)
Editor services for Angular templates
Disable Uninstall ⚙️
This extension is enabled globally.



Angular Snippets (Version 16) v16.0.1
John Papa [johnpapa.net](#) | 4,400,318 | ★★★★★ (96)
Angular version 16 snippets by John Papa
Uninstall ⚙️



Path Intellisense v2.8.5
Christian Kohler [christiankohler.net](#) | 12,684,657
Visual Studio Code plugin that autocompletes filenames
Uninstall ⚙️

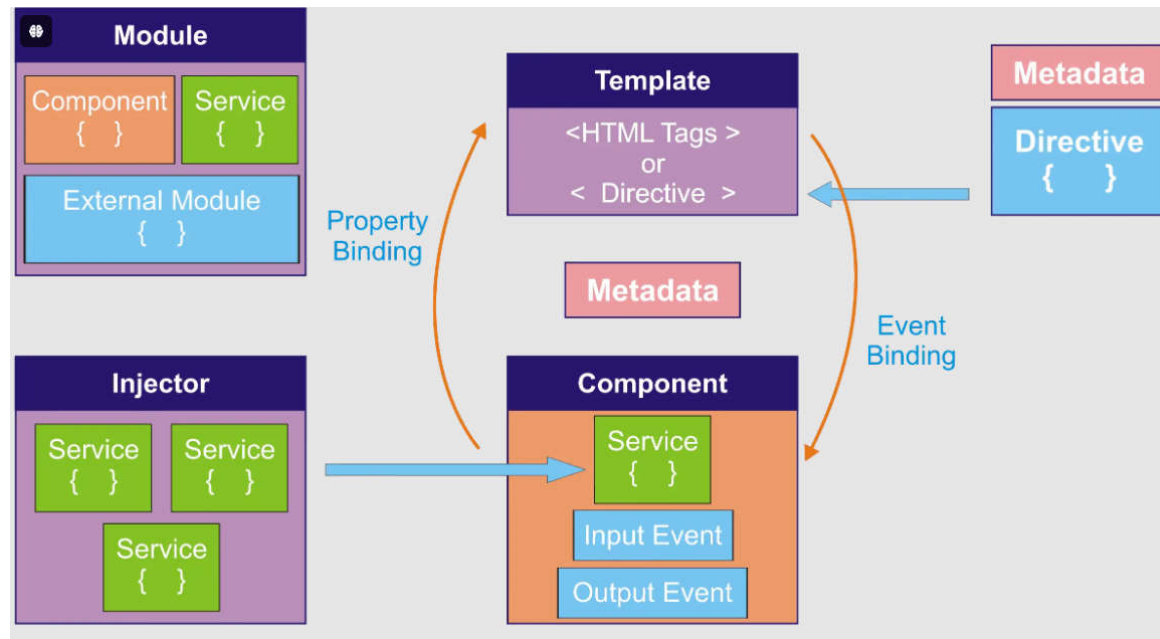
02

Création et structure d'un projet



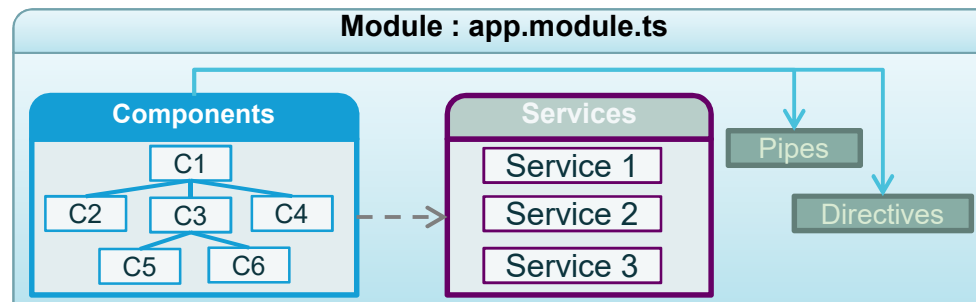
Architecture de base d'Angular

- **Angular** a une architecture **modulaire et composable** qui permet de créer des applications robustes et maintenables.



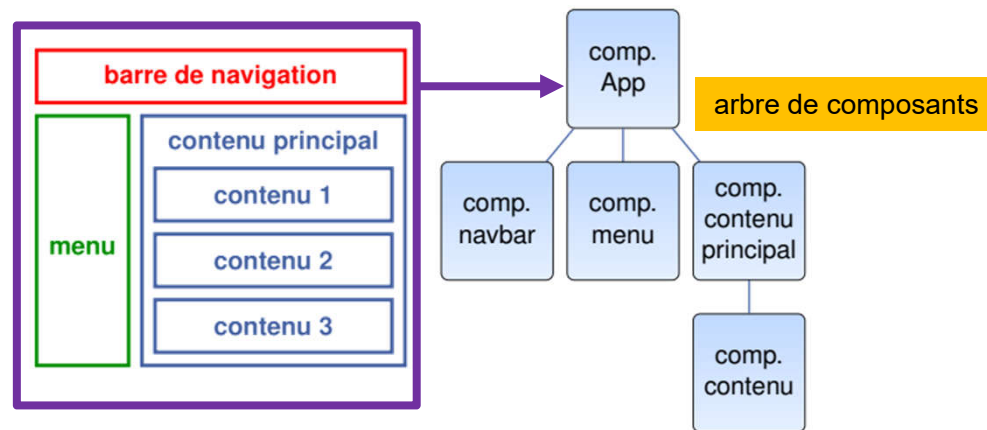
Architecture modulaire

- Une application **Angular** se compose de :
 - Un à plusieurs **modules** dont un est principal,
 - Un module représente un conteneur regroupant un ensemble d'élément d'une application,
 - Chaque **module** peut inclure :



- Des **composants** web : La partie visible de l'application Web (IHM)
- Des **services** pour la logique applicative : Les composants peuvent utiliser les services (`AuthService`, `CrudService`, ...) via le principe de **l'injection des dépendances**.
- Des **directives** : utilisées pour modifier l'apparence générale et la structure du DOM
- Des **pipes** : utilisés pour formater l'affichage des données dans les composants

Architecture Angular Component



- Angular repose sur les composants web (Web components)
- Dans la figure ci contre, chaque rectangle = un composant Angular
- Les composants sont réutilisables plusieurs fois
- Un composant peut en inclure d'autres

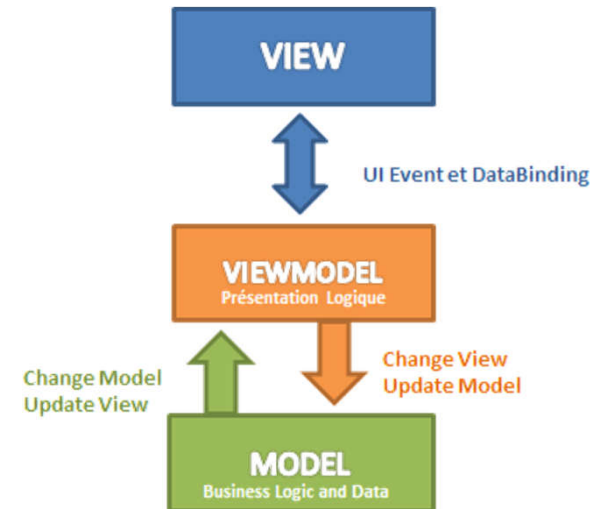
Contenu d'un composant Angular

Un composant Angular contient essentiellement :

- ❶ **un fichier TypeScript** (la classe du composant) contenant :
 - les données du composant
 - la logique/le comportement du composant
- ❷ **un fichier html** (le Template du composant) contenant :
 - le code HTML
 - des instructions pour interagir avec le code TypeScript
- ❸ **un fichier css** contenant le style propre au composant

Architecture MVVM

- **Angular** repose principalement sur l'architecture **MVVM(Model-View-ViewModel)** où :
 - **Model** : représente généralement les données de l'application (classe ou interface TypeScript) ainsi que la logique métier d'une application
 - **View** : Responsable de l'affichage des données à l'utilisateur.
 - Dans Angular, le Template d'un composant représentent la **vue**.
 - La **vue** déclenche des actions sur le **ModelView**,
 - Le **ViewModel** , qui remplace le contrôleur dans l'architecture MVC, est connecté à la vue par le **data binding**. Il est représenté dans Angular par la classe d'un composant (*.component.ts) et il expose les données et le comportement nécessaires à la vue



Architecture MVVM : Example

```
1 // Here's a simple example of MVVM implementation in Angular:
2 // Model
3 export class User {
4   id: number;
5   name: string;
6   email: string;
7 }
8 // Service : Contains reusable business logic and communication with external data sources. Used to encapsulate
9 // logic that doesn't directly belong in components, such as HTTP API calls, local storage,.
10 import { Injectable } from '@angular/core';
11 import { HttpClient } from '@angular/common/http';
12 import { Observable } from 'rxjs';
13 @Injectable({
14   providedIn: 'root'
15 })
16 export class UserService {
17   constructor(private http: HttpClient) {}
18
19   getUser(): Observable<User> {
20     // Assuming the API endpoint returns a user object
21     return this.http.get<User>('api/user');
22   }
23 }
```

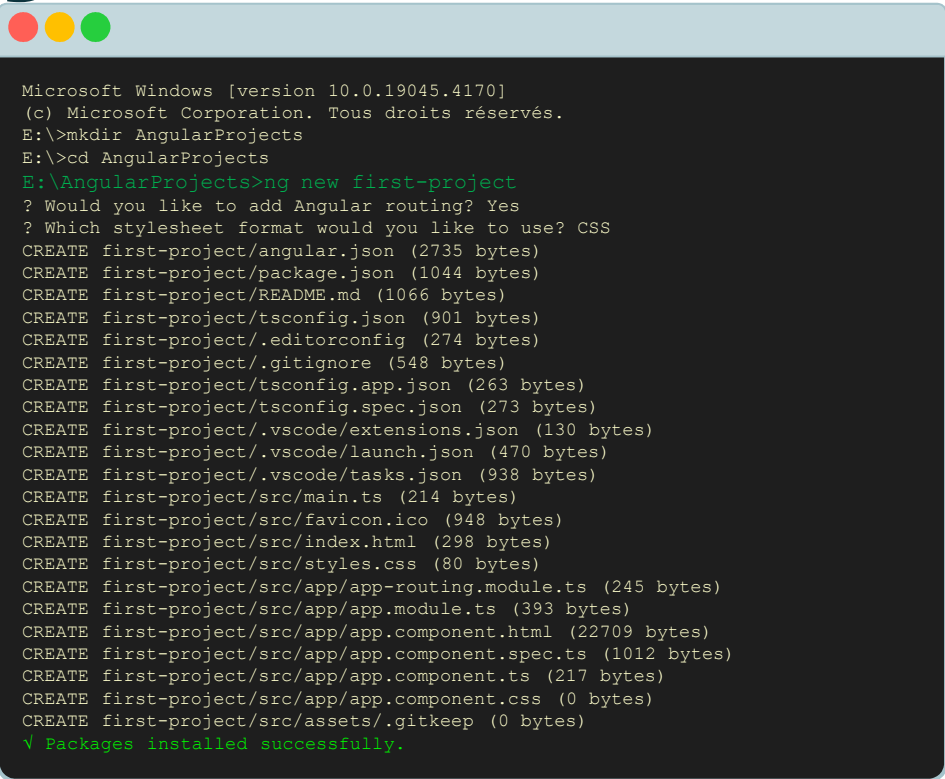
Architecture MVVM : Example

```
24 // View (Angular component: HTML template)
25 <div>
26   <p>{{ user?.name }}</p>
27   <p>{{ user?.email }}</p>
28 </div>
29
30 // ViewModel (Angular component : TypeScript file )
31 import { Component, OnInit } from '@angular/core';
32 import { User } from './user.model';
33 import { UserService } from './user.service';
34
35 @Component({
36   selector: 'app-user',
37   templateUrl: './user.component.html',
38   styleUrls: ['./user.component.css']
39 })
40 export class UserComponent implements OnInit {
41   user: User;
42
43   constructor(private userService: UserService) { }
44
45   ngOnInit(): void {
46     this.userService.getUser().subscribe(user => {
47       this.user = user;
48     });
49   }
50 }
```


Créer un projet Angular

- À un emplacement pratique sur votre ordinateur, créez un dossier nommé **AngularProjects** et accédez à ce dossier à l'aide de l'invite de commandes.
- Pour créer une nouvelle application nommée **first-project**, il suffit d'exécuter la commande suivante :

ng new first-project



```
Microsoft Windows [version 10.0.19045.4170]
(c) Microsoft Corporation. Tous droits réservés.
E:\>mkdir AngularProjects
E:\>cd AngularProjects
E:\AngularProjects>ng new first-project
? Would you like to add Angular routing? Yes
? Which stylesheet format would you like to use? CSS
CREATE first-project/angular.json (2735 bytes)
CREATE first-project/package.json (1044 bytes)
CREATE first-project/README.md (1066 bytes)
CREATE first-project/tsconfig.json (901 bytes)
CREATE first-project/.editorconfig (274 bytes)
CREATE first-project/.gitignore (548 bytes)
CREATE first-project/tsconfig.app.json (263 bytes)
CREATE first-project/tsconfig.spec.json (273 bytes)
CREATE first-project/.vscode/extensions.json (130 bytes)
CREATE first-project/.vscode/launch.json (470 bytes)
CREATE first-project/.vscode/tasks.json (938 bytes)
CREATE first-project/src/main.ts (214 bytes)
CREATE first-project/src/favicon.ico (948 bytes)
CREATE first-project/src/index.html (298 bytes)
CREATE first-project/src/styles.css (80 bytes)
CREATE first-project/src/app/app-routing.module.ts (245 bytes)
CREATE first-project/src/app/app.module.ts (393 bytes)
CREATE first-project/src/app/app.component.html (22709 bytes)
CREATE first-project/src/app/app.component.spec.ts (1012 bytes)
CREATE first-project/src/app/app.component.ts (217 bytes)
CREATE first-project/src/app/app.component.css (0 bytes)
CREATE first-project/src/assets/.gitkeep (0 bytes)
√ Packages installed successfully.
```

Lancer le projet

Se déplacer dans le projet

Lancer le projet, **-open** ouvre le navigateur par défaut

```
E:\AngularProjects>cd first-project

E:\AngularProjects\first-project>ng serve --open
√ Browser application bundle generation complete.

Initial Chunk Files | Names          | Raw Size
vendor.js           | vendor         | 2.35 MB
polyfills.js        | polyfills      | 332.07 kB
styles.css, styles.js | styles        | 229.35 kB
main.js             | main           | 48.34 kB
runtime.js          | runtime        | 6.52 kB
                    | Initial Total  | 2.95 MB

Build at: 2024-04-15T15:55:13.855Z - Hash: db53525c7c236825 - Time: 11452ms

** Angular Live Development Server is listening on localhost:4200, open your browser on
http://localhost:4200/ **

√ Compiled successfully.
```

Info

Pour créer un projet sans les fichiers de test `.spec.ts`, utiliser la commande: `$ng new ProjectName --skip-tests=true`
La commande `ng-serve` ouvre un onglet dans votre navigateur par défaut à l'adresse: `http: // localhost: 4200.`

FirstProject

Welcome

first-project app is running!

Resources

Here are some links to help you get started:

Learn Angular >

<> CLI Documentation >

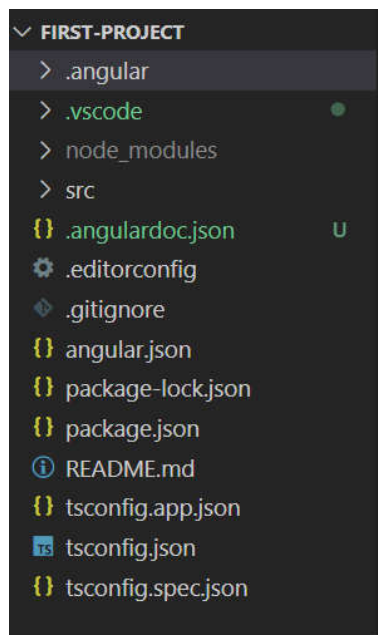
Angular Material >

Angular Blog >

Angular DevTools >

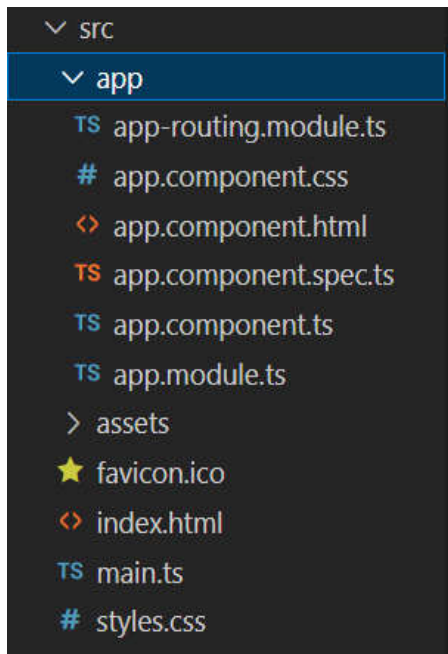
danouer.dahdeh@fsb.ucar.tn

Un projet Angular 16



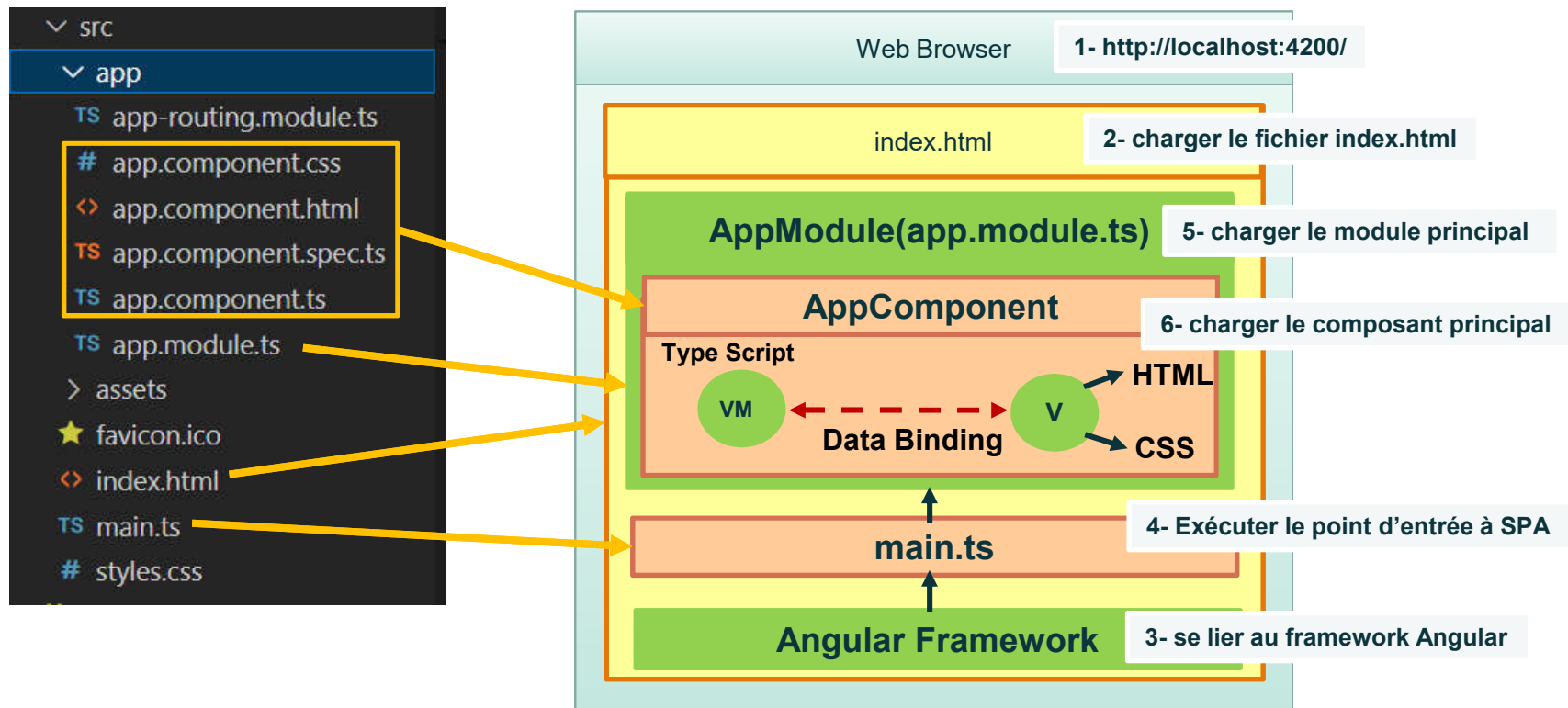
- **tsconfig.json**: la configuration TypeScript de base pour les projets dans le workspace
- **tsconfig.app.json**: le fichier de configuration TypeScript de l'application racine qui hérite de celui de base
- **tsconfig.spec.json** le fichier de configuration TypeScript des tests e2e qui hérite de celui de base
- **README.md**: documentation d'introduction pour l'application racine
- **package.json**: configure les dépendances des packages npm qui sont disponibles pour tous les projets de l'espace de travail
- **package-lock.json**: fournit des informations de version pour tous les packages installés dans node_modules par le client npm
- **angular.json**: Configuration du CLI, y compris des options de configuration pour les commandes build, serve et pour les outils de tests utilisés par le CLI
- **gitignore**: Spécifie les fichiers intentionnellement non suivis que Git doit ignorer
- **editorconfig**: Configuration pour les éditeurs de code
- **src**: Fichiers sources pour l'application
- **node_modules**: Fournit des packages npm à l'ensemble du workspace

Un projet Angular 16 – Dossier /src



- **assets** : Contient des images et d'autres fichiers d'assets à copier tels quels lors du build de votre application
- **favicon.ico** : Une icône à utiliser pour cette application dans la barre de favoris
- **index.html** : La page HTML principale qui est diffusée lorsqu'un internaute visite votre site. Le CLI ajoute automatiquement tous les fichiers JavaScript et CSS lors de la création de votre application, vous n'avez donc généralement pas besoin d'ajouter manuellement de balises `<script>` ou `<link>` ici.
- **styles.css** : La feuille de style commune de tous les composants web de l'application
- **main.ts** : Le point d'entrée principale de l'application
- **app** : Contient les fichiers de composants dans lesquels la logique et les données de votre application sont définies
 - **app.module.ts** : la classe correspondante au module principal
 - **app.component.ts** : la classe associée au composant web principal
 - **app.component.html** : le fichier contenant le code HTML associé au composant web principal
 - **app.component.css** : le fichier contenant le code CSS associé au composant web principal
 - **app.component.spec.ts** : le fichier de test du composant web principal

Etapes d'exécution d'une application Angular

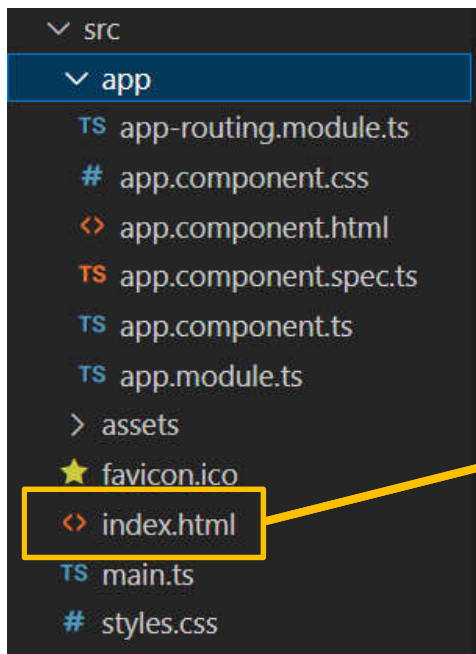


Scénario complet

1. La première page à charger, c'est la page **index.html**, ce fichier contient généralement une balise **<script>** qui charge le fichier JavaScript principal de l'application Angular, généralement nommé **main.js** ou **main.ts** s'il n'a pas encore été compilé,
2. Angular cherche **main.ts** (**main.js**),
3. À l'intérieur de **main.ts**, il cherche le module principal à déclencher (Dans notre cas: **AppModule**),
4. Il cherche le fichier de ce module (dans notre cas c'est **app.module.ts**),
5. À l'intérieur de **app.module.ts**, il cherche le composant principal à déclencher (dans notre cas c'est **AppComponent**),
6. Une fois que le composant principal est initialisé, Angular remplace la balise **<app-root>** dans **index.html** avec le contenu du composant **AppComponent**. Ceci est possible grâce à la propriété **selector** dans la configuration du composant.

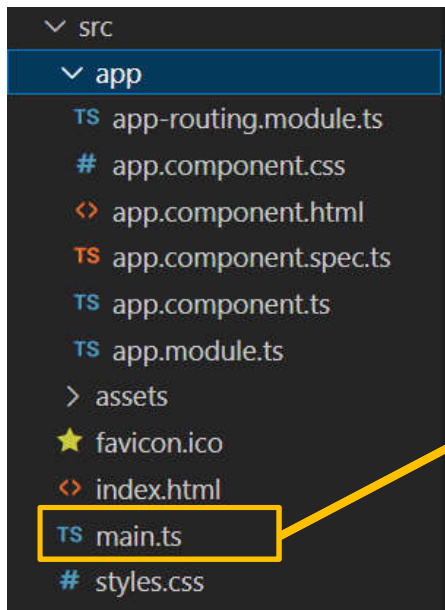
... tout ça donne le rendu visuel final !

index.html : Première et dernière page HTML à charger



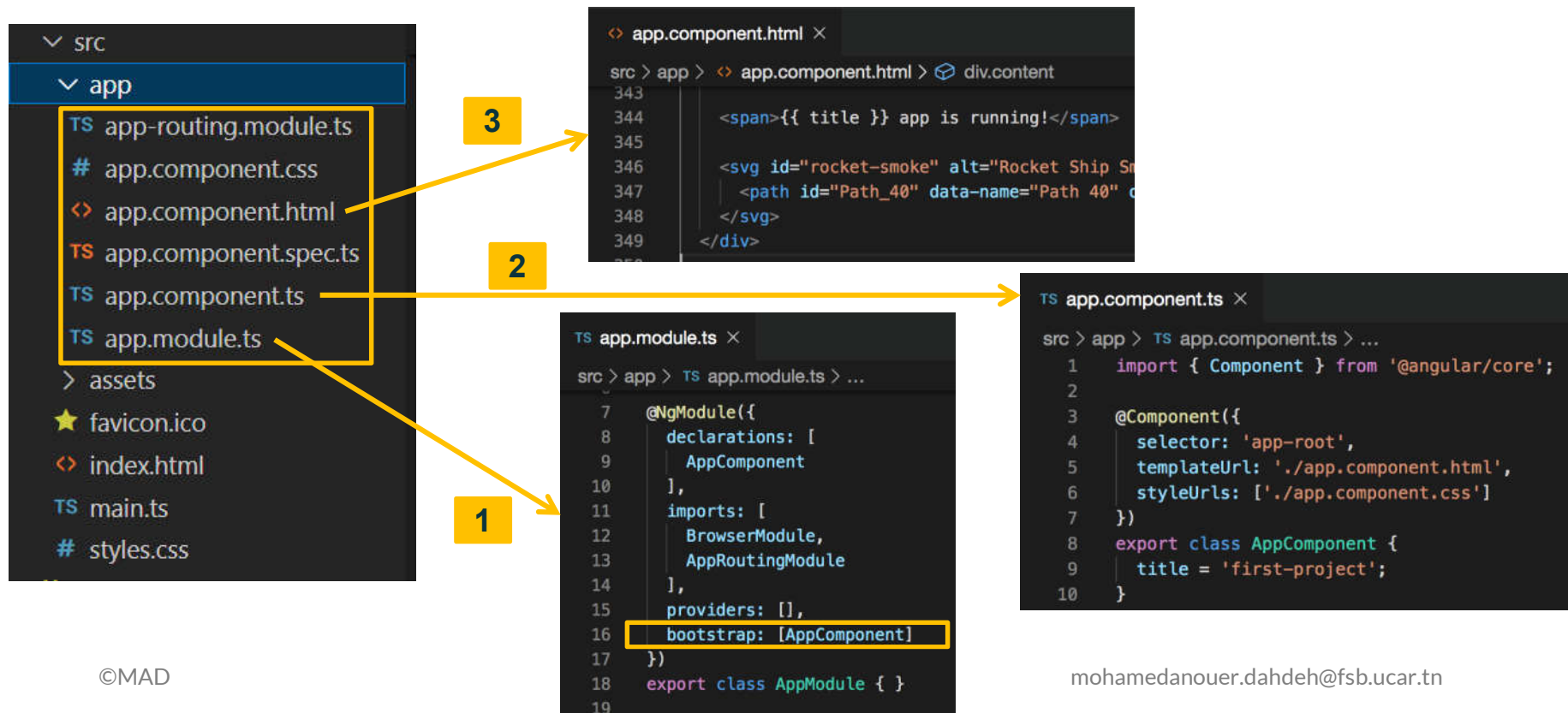
```
<> index.html ×
src > <> index.html > html > body
1  <!doctype html>
2  <html lang="en">
3  <head>
4    <meta charset="utf-8">
5    <title>FirstProject</title>
6
7    <!-- balise permettant le routage -->
8    <base href="/">
9    <meta name="viewport" content="width=device-width, initial-scale=1">
10   <link rel="icon" type="image/x-icon" href="favicon.ico">
11 </head>
12 <body>
13
14   <!-- le composant web app.component introduit dans le module -->
15   <app-root></app-root>
16 </body>
17 </html>
18
```

main.ts : le point d'entrée à l'application



```
TS main.ts  X
src > TS main.ts > ...
1  import { enableProdMode } from '@angular/core';
2  import { platformBrowserDynamic } from '@angular/platform-browser-dynamic';
3
4  import { AppModule } from './app/app.module';
5  import { environment } from './environments/environment';
6
7  if (environment.production) {
8    enableProdMode();
9  }
10
11  platformBrowserDynamic().bootstrapModule(AppModule)
12    .catch(err => console.error(err));
```


Un projet Angular 16 – /app



Module et @NgModule

Modules

- Les applications **Angular** sont modulaires
- Un module Angular regroupe un ensemble de composants Angular dans une même unité logique
- Tout comme les classes sont utilisées pour regrouper des méthodes et des propriétés dans la programmation orientée objet, les modules Angular sont utilisés pour regrouper des fonctionnalités.
- Angular possède son propre système de modularité appelé **NgModule**.
- Chaque application Angular possède au moins une classe Module : le module principal **AppModule**.

@NgModule

- Un module est une classe avec un décorateur **@NgModule**.
- Les décorateurs permettent de décrire au framework comment interpréter un élément.
- Angular possède de nombreux décorateurs qui attachent des **métadonnées aux classes** pour configurer et donner du sens à ces classes.
- **@NgModule** est un décorateur qui prend en paramètre un objet javascript qui contient des métadonnées dont les propriétés décrivent un module.

App.module.ts

@NgModule pour déclarer
cette classe comme
module

déclarer le
composant racine

```
TS app.module.ts x
src > app > TS app.module.ts > ...
1  import { BrowserModule } from '@angular/platform-browser';
2  import { NgModule } from '@angular/core';
3
4  import { AppRoutingModule } from './app-routing.module';
5  import { AppComponent } from './app.component';
6
7  @NgModule({
8    declarations: [
9      AppComponent
10   ],
11   imports: [
12     BrowserModule,
13     AppRoutingModule
14   ],
15   providers: [],
16   bootstrap: [AppComponent]
17 })
18 export class AppModule { }
19
```

déclarer les composants de ce module

déclarer les modules nécessaires pour ce module

déclarer les créateurs de services nécessaires
pour ce module

Les fichiers du composant app

The diagram illustrates the file structure of an Angular component named 'app'. The file explorer on the left shows the following files in the 'src/app' directory:

- TS app-routing.module.ts
- # app.component.css
- <> app.component.html
- TS app.component.spec.ts
- TS app.component.ts
- TS app.module.ts
- > assets
- ★ favicon.ico
- <> index.html
- TS main.ts
- # styles.css

Arrows indicate the mapping from the file explorer to the code snippets:

- app.component.css**:

```
# app.component.css ×
src > app > # app.component.css
1
```
- app.component.html**:

```
<> app.component.html ×
src > app > <> app.component.html > div.content
343
344 <span>{{ title }} app is running!</span>
345
346 <svg id="rocket-smoke" alt="Rocket Ship Smoke" >
347   <path id="
348 </svg>
349 </div>
```
- app.component.ts**:

```
TS app.component.ts ×
src > app > TS app.component.ts > ...
1 import { Component } from '@angular/core';
2
3 @Component({
4   selector: 'app-root',
5   templateUrl: './app.component.html',
6   styleUrls: ['./app.component.css']
7 })
8 export class AppComponent {
9   title = 'first-project';
10 }
```

Les fichiers des tests unitaires. La convention est d'avoir un fichier .spec.ts pour chaque fichier .ts. Ils sont exécutés à l'aide du framework de test Jasmine via le programme de tâches Karma lors de l'exécution de la commande « ng test »

first-project app is running!

sb.ucar.tn

Components

```
TS app.component.ts ×
src > app > TS app.component.ts > ...
1  import { Component } from '@angular/core';
2
3  @Component({
4    selector: 'app-root',
5    templateUrl: './app.component.html',
6    styleUrls: ['./app.component.css']
7  })
8  export class AppComponent {
9    title = 'first-project';
10 }
```

```
<> app.component.html ×
src > app > <> app.component.html > div.content
343
344   <span>{{ title }} app is running!</span>
345
346   <svg id="rocket-smoke" alt="Rocket Ship Smoke" >
347     <path id="Path_40" data-name="Path 40" d="
348   </svg>
349 </div>
```



first-project app is running!

Explication

- **@Component** : un décorateur pour déclarer cette classe comme composant web
- **selector** : pour définir le nom de balise correspondant à ce composant web
- **templateUrl** : indique le fichier contenant le code HTML du composant
- **styleUrls** : indique le(s) fichier(s) contenant le code CSS

Remarque : interpolation (une forme de data binding)

`{{ title }}` : title sera remplacé par sa valeur dans `app.component.ts`

03

Atelier 1

